LINQ — Exerciții

Stefan · Partea 3 · 5 exerciții + provocare, în consolă · doar LINQ

Repo-ul: cel de la partea 3. Soluția `CollectionsLinq.sln` are proiectul `Exercitii/` (.NET 8, `LangVersion` 7.3 în csproj) și `exemplu/ExempluLinq/`. Scrii în `Exercitii/Exercitii.cs`, în clasa `Exercitii`: fiecare exercițiu = metoda `public static void ExN()` (există deja, goală), cu codul din „În `ExN`" copiat ca atare, iar metodele din enunț stau lângă ea. `Main` din `Program.cs` cheamă deja `Ex1()` … `Ex6()`, cu `--- N` între ele. Ieșirea fiecărui exercițiu trebuie să fie **identică** cu cea din fișă.
Clasele: `Elev` și `Catalog()` sunt deja în `Exercitii.cs`. `using System.Linq;` și celelalte `using`-uri le scrii tu. Prima linie din `Main` setează deja cultura invariantă, ca zecimalele să iasă cu punct.
Reguli: C# 7.3 și doar metode care există în .NET Framework 4.x (fără `DistinctBy`, `MinBy`; în loc de `MinBy`, `OrderBy(...).First()`) · **doar LINQ**, fără `for` / `foreach` în metodele tale · un exercițiu per commit.
Exemplul: `exemplu/ExempluLinq/`, cu 3 secțiuni; ieșirea lui e în TEORIE.
Ex3 pornește de la codul tău din partea 2: repo-ul `collections-dotnet`, fișierele `Exercitii/Sectiune3ISet.cs` și `Exercitii/Sectiune6Alegere.cs`. **Ex6** pornește de la metoda dată `CelMaiFrecventCuvant`, din `Exercitii.cs`.

1. Premianții

`static List<string> Premianti(List<Elev> elevi)`: elevii cu media cel puțin 9, descrescător după medie, iar la medii egale alfabetic. Fiecare rând: nume, spațiu, media cu două zecimale.

În `Ex1`:

```
List<Elev> elevi = Catalog();
elevi.Reverse();
foreach (string linie in Premianti(elevi))
{
    Console.WriteLine(linie);
}
```

Ieșire așteptată:

```
Maria 9.80
Ana 9.50
Vlad 9.50
Dan 9.10
```

Teorie: §1 · `Where`, `OrderByDescending`, `ThenBy`, `Select`, `ToList`

2. Raportul pe clase

`static List<string> RaportClase(List<Elev> elevi)`: câte un rând pe clasă, în ordinea claselor, cu numărul de elevi, media clasei cu două zecimale și elevul cu media cea mai mare.

În `Ex2`:

```
foreach (string linie in RaportClase(Catalog()))
{
    Console.WriteLine(linie);
}
```

Ieșire așteptată:

```
10C: 2 elevi, media 8.45, primul Vlad
11B: 2 elevi, media 8.65, primul Dan
12A: 3 elevi, media 8.07, primul Maria
```

Teorie: §1 · `GroupBy` ; un grup e el însuși o colecție, deci merge `g.Count()` , `g.Average(...)`

3. Rescrie cu LINQ metode din partea 2

Câte o variantă LINQ, cu sufixul `Linq` în nume, pentru patru metode din partea 2 (repo-ul `collections-dotnet`): `SituatiaVizitatori` (6.1), `ReuniuneCuIonSiEva` (3.2), `IntersectieCuIonMariaZoe` (3.3) și `SituatiaClasament` (6.10). Fiecare e o metodă `static void` care pornește de la aceleași date și afișează același lucru (`SituatiaVizitatoriLinq` …). `Ex3` le cheamă pe rând, în ordinea de mai sus.

Ieșire așteptată:

```
3
Ana, Ion, Maria, Eva
Ion, Maria
Ana=9, Ion=7, Zoe=5
```

Gata când: ieșirea e identică cu cea din partea 2 la 6.1, 3.2, 3.3 și 6.10. În `RASPUNSURI.txt` : la fiecare pereche, care variantă ți se pare mai ușor de citit și de ce?

Teorie: §1 · `Distinct` , `Union` , `Intersect` , `Select`

4. Întrebări despre catalog

În `Ex4` :

```
List<Elev> cat = Catalog();
Console.WriteLine(___1___);
Console.WriteLine(___2___);
Console.WriteLine(___3___);
Elev lipsa = ___4___;
Console.WriteLine(lipsa == null ? "null" : lipsa.Nume);
Console.WriteLine(___5___);
try
{
    Console.WriteLine(___6___);
}
catch (InvalidOperationException ex)
{
    Console.WriteLine(ex.GetType().Name);
}
```

Fiecare `___n___` e o singură expresie LINQ: **1** există vreun elev cu media sub 5? · **2** toți au media cel puțin 5? · **3** numele primului elev din 11B · **4** primul elev din 9A, fără să arunce · **5** cea mai mare medie, cu două zecimale · **6** numele primului elev din 9A, cu `First` .

Ieșire așteptată:

```
True
False
Ion
null
9.80
InvalidOperationException
```

Teorie: §1 · `Any` , `All` , `First` , `FirstOrDefault` , `Max`

5. Rețeta, nu rezultatul

În `Ex5`:

```
List<int> valori = new List<int> { 5, 20, 8, 30 };
IEnumerable<int> mari = valori.Where(v => v > 10);
List<int> mariAcum = valori.Where(v => v > 10).ToList();
valori.Add(50);
Console.WriteLine(string.Join(" ", mari));
Console.WriteLine(string.Join(" ", mariAcum));
```

Înainte să rulezi, scrie în `RASPUNSURI.txt` ce crezi că afișează. Apoi rulează și explică diferența dintre cele două rânduri.

Teorie: §2

Provocare

6. Cuvântul cel mai des întâlnit, cu LINQ

În `Exercitii.cs` ai deja `CelMaiFrecventCuvant(string text)`, scrisă cu bucle: cuvintele se despart după spațiu, se scriu cu litere mici și fără `. , ! ?` la capete, iar la frecvență egală câștigă cel mai mic alfabetic (`StringComparer.Ordinal`). Scrie `static string CelMaiFrecventCuvantLinq(string text)`, cu aceleași reguli, fără nicio buclă.

În `Ex6`:

```
string text = "Ana are mere. Ion are pere, ana are si prune!";
Console.WriteLine(CelMaiFrecventCuvantLinq(text));
Console.WriteLine(CelMaiFrecventCuvantLinq("mar par mar par"));
Console.WriteLine(CelMaiFrecventCuvantLinq("par mar par mar"));
```

Ieșire așteptată:

```
are
mar
mar
```

Gata când: aceleași trei texte date lui `CelMaiFrecventCuvant` (chemat o dată de probă, apoi scos) dau aceleași trei rezultate.

Teorie: §1 · `GroupBy`, `OrderByDescending`, `ThenBy`, `First`